COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	September 18 , 2026	**DATE SUBMITTED:**	September 23,2026
NAME:	Macasaet John David C.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. ___ HOA 9.1 Tree ADT

1. PROCEDURE OUTPUT

Code Snippet

generalTree.h

```
template <typename T>
class Node{
    public:
        T key;
        Node* firstChild;
        Node* nextSibling;

    Node(T data){
        key = data;
        firstChild = nullptr;
        nextSibling = nullptr;
    }
};

template <typename T>
void displayRoot(Node<T>* root){
    if(root != nullptr){
        std::cout<<"Root Node: "<<root->key<<std::endl;
    }
}
```

```cpp
template <typename T>
void setRoot(Node<T>*& rootPtr, T val){
    rootPtr = new Node<T>(val);
}
//findNode function so that you don't have to change the value of a root
//everytime you want to create children of a child of root.
template <typename T>
Node<T>* findNode(Node<T>* current, T target) {
    if (current == nullptr){
        return nullptr;
    }
    if (current->key == target){
        return current;
    }

    Node<T>* found = findNode(current->firstChild, target);
    if (found != nullptr){
        return found;
    }

    return findNode(current->nextSibling, target);
}
```

```cpp
template <typename T>
void addChild(Node<T>* root, T parent, T child){
    Node<T>* newNode = new Node<T>(child);
    Node<T>* parentNode = findNode(root, parent);
    if(parentNode != nullptr){
        if(parentNode->firstChild == nullptr){
        parentNode->firstChild = newNode;
        }else{
            Node<T>* temp = parentNode->firstChild;
            while(temp->nextSibling != nullptr){
                temp = temp->nextSibling;
            }
            temp->nextSibling = newNode;
        }
    }
}
```

```
template <typename T>
void displayTree(Node<T>* root, T target) {
    Node<T>* targetNode = findNode(root, target);
    if (targetNode == nullptr) {
        std::cout << "Node " << target << " not found." << std::endl;
        return;
    }

    std::cout << "Children of Node " << targetNode->key << ": ";
    Node<T>* current = targetNode->firstChild;
    while (current != nullptr) {
        std::cout << current->key << " ";
        current = current->nextSibling;
    }
    std::cout << std::endl;
}
```

Main.cpp

```
//GENERAL TREE
//creates the children of root A
Node<char>* root = new Node<char>('A');
displayRoot<char>(root);
addChild<char>(root, 'A', 'B');
addChild<char>(root, 'A','C');
addChild<char>(root, 'A','D');
addChild<char>(root, 'A', 'E');
addChild<char>(root, 'A', 'F');
addChild<char>(root, 'A', 'G');
displayTree<char>(root, 'A');
//creates the children of node D
addChild<char>(root, 'D', 'H');
displayTree<char>(root, 'D');
//creates the children of node E
addChild<char>(root,'E', 'I');
addChild<char>(root,'E', 'J');
displayTree<char>(root, 'E');
//creates the children of node F
addChild<char>(root,'F', 'K');
addChild<char>(root,'F', 'L');
addChild<char>(root,'F', 'M');
displayTree<char>(root, 'F');
//creates the children of node G
addChild<char>(root,'G', 'N');
displayTree<char>(root, 'G');
```

```
//creates the children of node J
addChild<char>(root,'J', 'P');
addChild<char>(root,'J', 'Q');
displayTree<char>(root, 'J');
```

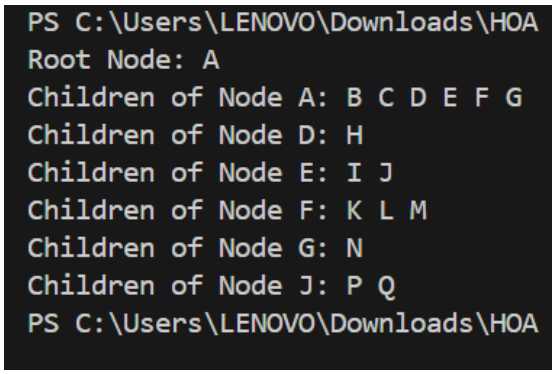
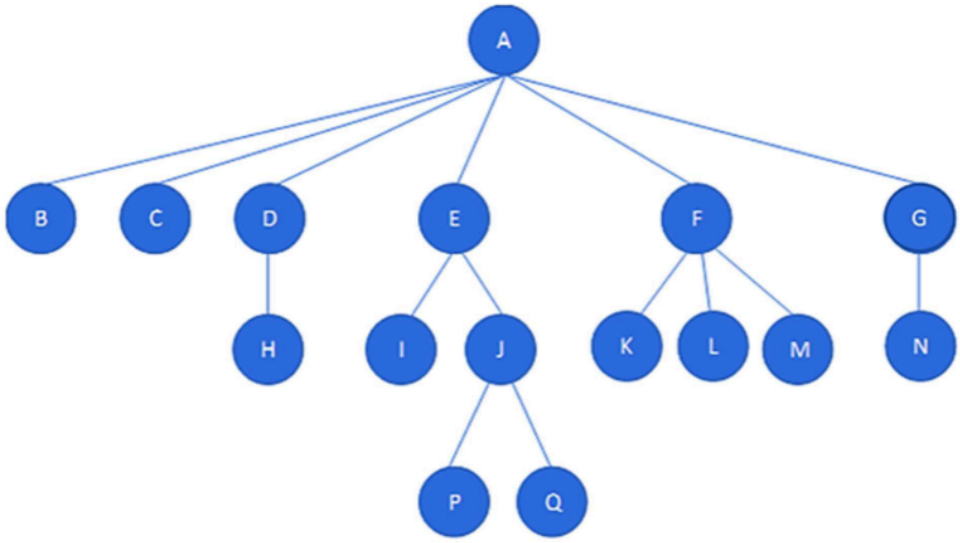
Output	
Observation	General Tree differs from binary tree in that binary tree has only a maximum of two children per node. Whereas in a general tree, a root or node can have many more children.

Table 9.1: ILO A Task 1

NODE	HEIGHT	DEPTH
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2

N	0	2
P	0	3
Q	0	3

Table 9.2: ILO A Task 2



Pre-Order	A->B->C->D->H->E->I->J->P->Q->F->K->L->M->G->N
Post-Order	B->C->H->D->I->P->Q->J->E->K->L->M->F->N->G->A
In-Order	B->C->H->D->I->P->Q->J->E->K->L->M->F->N->G->A

Table 9.3

PRE-ORDER

```
template <typename T>
void preOrder(Node<T>* current) {
    if (current == nullptr) return;

    std::cout << current->key << " ";

    preOrder(current->firstChild);

    preOrder(current->nextSibling);
}
```

```
Pre-order: A B C D H E I J P Q F K L M G N
```

Observation: In a pre-order, it follows the ROOT-LEFT-RIGHT rule. It first goes to the root node, then traverses from left to right. My output here perfectly matches the output in Table 9.3

POST-ORDER

```
template <typename T>
void postOrder(Node<T>* current) {
    if (current == nullptr) return;

    postOrder(current->firstChild);

    std::cout << current->key << " ";

    postOrder(current->nextSibling);
}
```

```
Post-order: B C H D I P Q J E K L M F N G A
```

Observation: In a post-order, it follows the LEFT-RIGHT-ROOT rule. It first goes to the left node, then traverses to the right and then the root. My output here perfectly matches the output in Table 9.3

IN-ORDER

```cpp
template <typename T>
void inOrder(Node<T>* current) {
    if (current == nullptr) return;

    inOrder(current->firstChild);

    std::cout << current->key << " ";

    inOrder(current->nextSibling);
}
```

```
In-order: B C H D I P Q J E K L M F N G A
```

Observation: In in-order, it follows the LEFT-ROOT-RIGHT rule. It first goes to the left node, then traverses to root then right. My output here perfectly matches the output in Table 9.3

Table 9.4

Code Snippet

generalTree.h

```cpp
template <typename T>
bool searchPreOrder(Node<T>* current, T key) {
    if (current == nullptr){
        return false;
    }
    if (current->key == key){
        return true;
    }
    if (searchPreOrder(current->firstChild, key)){
        return true;
    }
    return searchPreOrder(current->nextSibling, key);
}
```

```cpp
template <typename T>
bool searchInOrder(Node<T>* current, T key) {
    if (current == nullptr){
        return false;
    }
    if (searchInOrder(current->firstChild, key)){
        return true;
    }
    if (current->key == key){
        return true;
    }
    return searchInOrder(current->nextSibling, key);
}
```

```
template <typename T>
bool searchPostOrder(Node<T>* current, T key) {
    if (current == nullptr){
        return false;
    }
    if (searchPostOrder(current->firstChild, key)){
        return true;
    }
    if (searchPostOrder(current->nextSibling, key)){
        return true;
    }
    return current->key == key;
}
```

```
template <typename T>
void findData(Node<T>* root, int choice, T key) {
    bool found = false;

    switch (choice) {
        case 1: // Pre-order
            found = searchPreOrder(root, key);
            break;
        case 2: // In-order
            found = searchInOrder(root, key);
            break;
        case 3: // Post-order
            found = searchPostOrder(root, key);
            break;
        default:
            std::cout << "Invalid choice!" << std::endl;
            return;
    }

    if (found) {
        std::cout << "Node " <<key << " was found!" << std::endl;
    }
}
```

main .cpp

```
findData<char>(root, 1, 'G');
findData<char>(root, 2, 'D');
findData<char>(root, 3, 'F');
```

Output

```
Node G was found!
Node D was found!
Node F was found!
```

Observation

Creating separate search functions for each traversal method allows us to search for a key using different methods. The findData function allows us to choose between traversal methods using switch-case.

Table 9.5

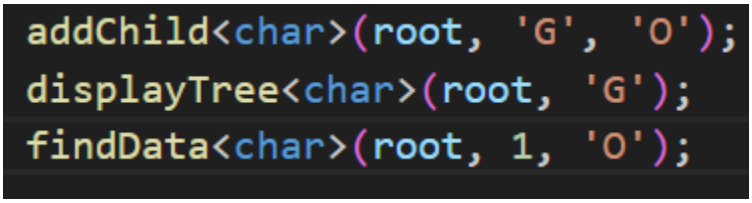
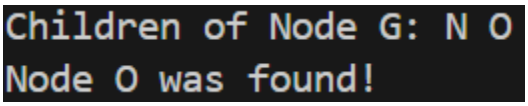
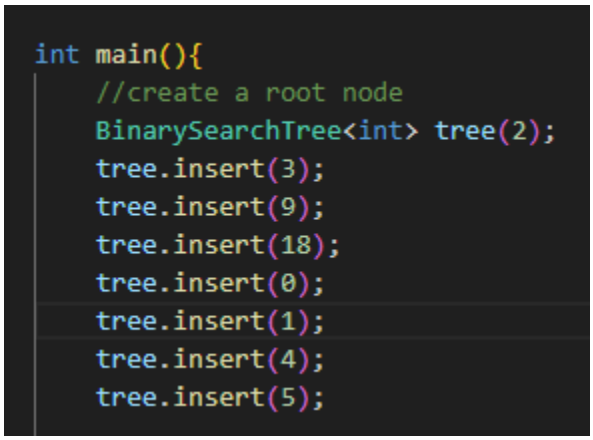
Code Snippet	
Output	
Observation	In this, using the addChild function I manage to add O as a child of G. And using findData I chose choice 1 which is pre-order traversal and using that Node O was found.

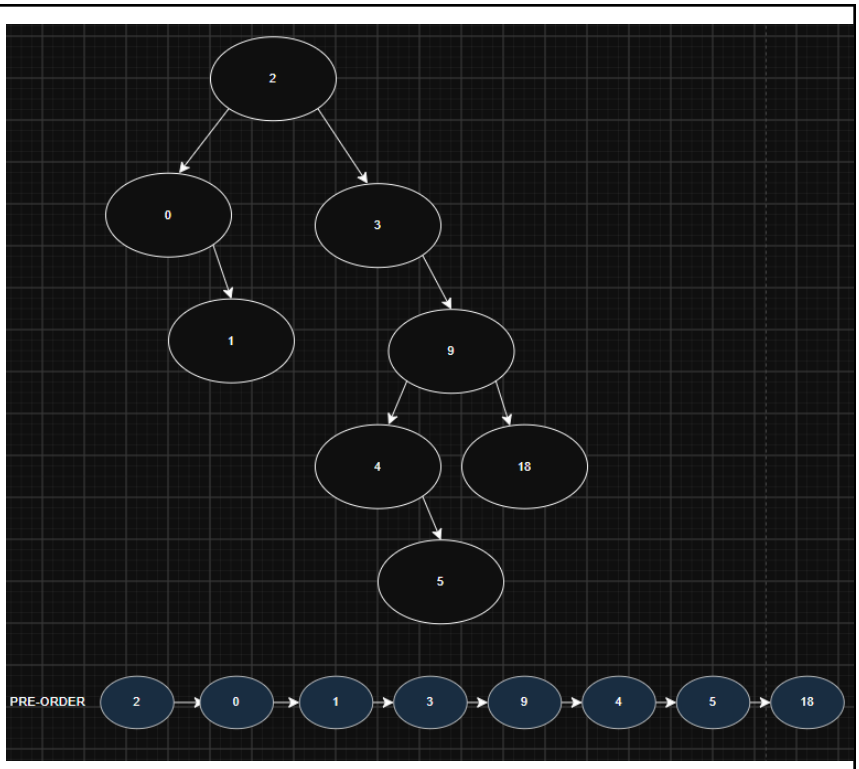
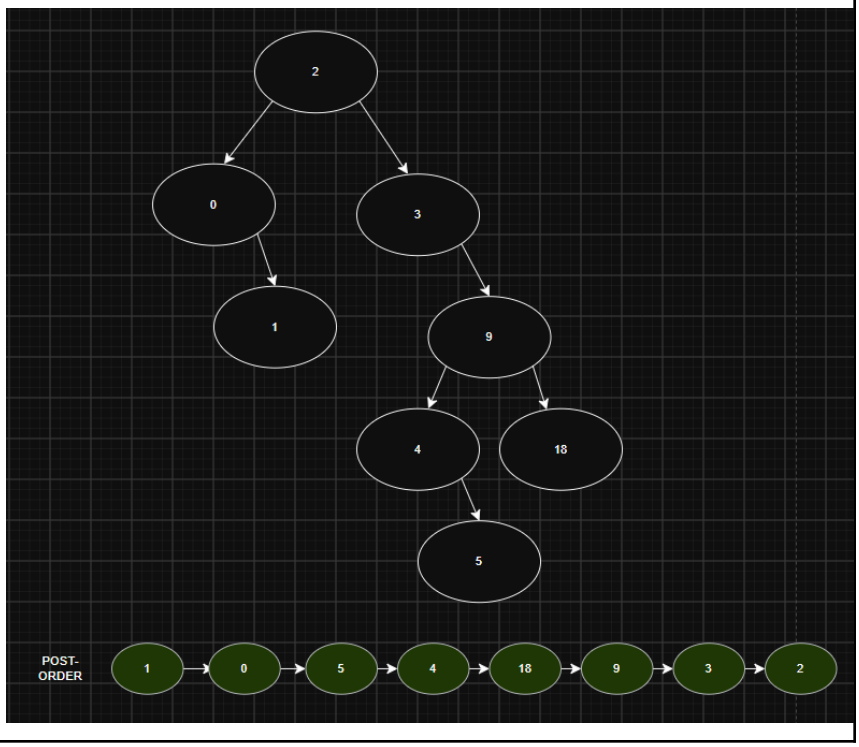
Table 9.6

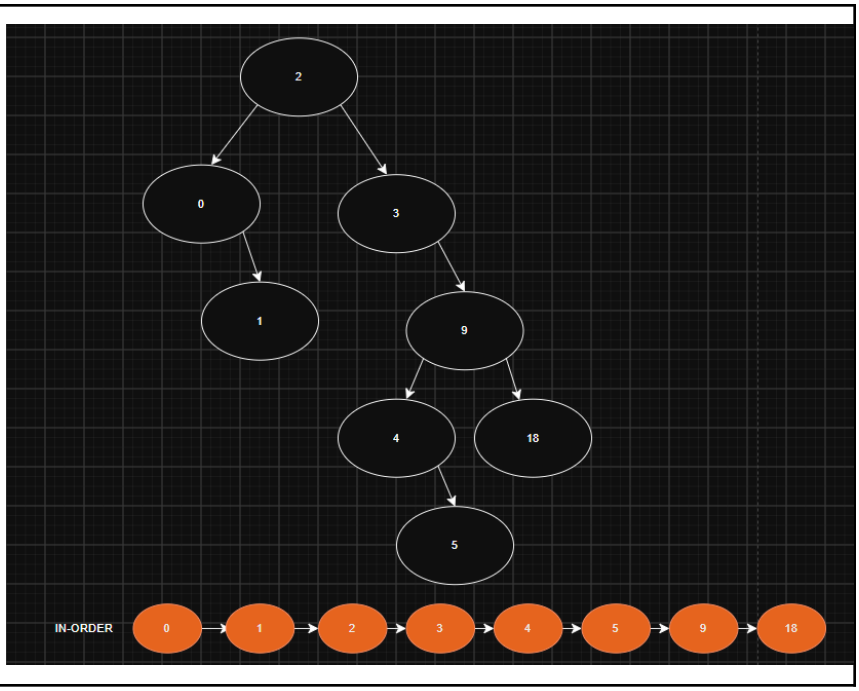
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

STEP 1:



STEP 2:

Pre-Order	
Post-Order	

In-Order	

STEP 3:

IN-ORDER

```
template <typename T>
void BinarySearchTree<T>::inOrder(){
    //go to left
    if(left != nullptr){
        left->inOrder();
    }
    std::cout<< key <<" ";
    //go to right
    if(right != nullptr){
        right->inOrder();
    }
}
```

```
inOrder: 0 1 2 3 4 5 9 18
```

The output console shows the same result to the in-order diagram I did in step 2

PRE-ORDER

```
template <typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout << key <<" ";
    if(left != nullptr){
        left->preOrder();
    }

    if(right != nullptr){
        right->preOrder();
    }

}
```

```
preOrder: 2 0 1 3 9 4 5 18
```

The output console shows the same result to the pre-order diagram I did in step 2

POST-ORDER

```
template <typename T>
void BinarySearchTree<T>::postOrder(){
    //go to left
    if(left != nullptr){
        left->postOrder();
    }
    //go to right
    if(right != nullptr){
        right->postOrder();
    }
    std::cout<< key <<" ";
}
```

```
postOrder: 1 0 5 4 18 9 3 2
```

The output console shows the same result to the post-order diagram I did in step 2

CONCLUSION: In this activity, I managed to implement Binary Tree and General Tree and how to traverse it using three methods which are post-order, pre-order, and in-order. One of the most important things I learned is the difference between a general and binary tree. A binary has a maximum of two children, while a general tree can have more than that. This activity further enhanced my knowledge on trees and its kinds and how it can be implemented.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **Idea generation, code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I ask it for guidance but not the full code, I try to interpret its guide and I translate it into code.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.